

Taller: procesos, scripts y llamadas al sistema

Laboratorio de Sistemas Operativos

Objetivo: observar los procesos vivos del sistema desde el shell y desde un programa propio, y resolver con un script una tarea de administración que no es práctica de hacer a mano. Al terminar se entregan un script que informa sobre un proceso, un programa en C que obtiene la misma información con llamadas al sistema, un script de conversión por lotes y una bitácora que explica por qué la información coincide.

Punto de partida

Se supone resuelto: compilar y ejecutar un programa en C desde la terminal, y moverse por el sistema de archivos. Quien entregue por repositorio usa el que quedó configurado en la sesión 3.

Se supone leído el texto guía *El shell y los scripts de shell*, en particular parámetros posicionales, sustitución de comandos, operadores de patrones, `if` y `while`.

El taller se resuelve dentro de la máquina virtual, donde la cuenta de trabajo tiene privilegios de administrador. Ninguna parte del enunciado modifica la configuración del sistema anfitrión.

Preparación: un usuario propio dentro de la máquina virtual

El trabajo del taller se hace desde un usuario creado para esta práctica, cuyo nombre es el login institucional de quien lo resuelve. Las capturas de pantalla que se entregan se toman desde una sesión de ese usuario, de modo que el símbolo del sistema muestre su nombre.

Cuando el taller se resuelve en pareja, cada integrante crea su propio usuario, con su propio login, en la máquina virtual desde la que trabaja.

En los ejemplos que siguen, `mi login` representa ese nombre.

1. Crear el usuario, asignarle una contraseña y agregarlo al grupo `sudo`, que es el que habilita la ejecución de tareas administrativas. La quinta parte las necesita:

```
$ sudo adduser milogin
$ sudo adduser milogin sudo
```

2. Comprobar que la cuenta quedó registrada, leyendo su línea en el archivo de cuentas del sistema, y comprobar a qué grupos pertenece. Identificar en la primera salida el identificador de usuario, el directorio de inicio y el intérprete asignado:

```
$ grep milogin /etc/passwd
$ groups milogin
```

3. Abrir una sesión del usuario recién creado y comprobar que el símbolo del sistema muestra su nombre:

```
$ su - milogin
$ whoami
```

El sistema de archivos /proc

El sistema operativo expone el estado de cada proceso vivo como archivos de texto bajo el directorio /proc. Cada proceso tiene un subdirectorio cuyo nombre es su identificador, y dentro de él, un archivo `status` con su información básica:

```
$ cat /proc/self/status
```

El nombre `self` se refiere al proceso que hace la lectura, de modo que la orden anterior muestra la información del propio `cat`. Para consultar otro proceso se usa su identificador en lugar de `self`. En el resto del enunciado, PID representa ese número.

Los archivos de /proc no ocupan espacio en disco: se generan en el momento de leerlos, con el estado del proceso en ese instante.

Primera parte: observación desde el shell

4. Listar los procesos del sistema y localizar en la salida las columnas de identificador, identificador del padre, estado y línea de comandos:

```
$ ps -ef
$ ps aux
```

5. Lanzar un proceso propio en segundo plano, anotar el identificador que informa el shell, y localizarlo en la lista de procesos:

```
$ sleep 300 &
$ ps -ef | grep sleep
```

6. Consultar el estado de ese proceso en `/proc` y comparar los campos `Name`, `State`, `PPid` y `Threads` con lo que informa `ps` sobre el mismo proceso.

Preguntar después por el tamaño de ese archivo de dos maneras distintas, y anotar en la bitácora lo que informa cada una:

```
$ stat -c %s /proc/self/status
$ wc -c < /proc/self/status
```

Las dos respuestas no coinciden. Explicar en la bitácora a qué se debe, teniendo en cuenta que `stat` consulta los atributos del archivo y `wc` lee su contenido.

7. Terminar el proceso lanzado en el punto 5 con `kill` y comprobar que su subdirectorio de `/proc` deja de existir.
8. Recorrer la cadena de padres a mano: partiendo del shell, consultar el campo `PPid`, consultar después el estado de ese padre, y repetir hasta llegar al proceso 1. Anotar en la bitácora la cadena completa y qué programa corresponde al proceso 1.

La orden `ps tree -p` muestra esa misma jerarquía de una sola vez. No está instalada en todas las distribuciones; si no responde, la cadena obtenida a mano es suficiente.

Segunda parte: el informe escrito como script de shell

9. Escribir un script llamado `infoproc.sh` que reciba un identificador de proceso como parámetro. Si no recibe ninguno, el script informa sobre el proceso que lo está ejecutando, cuyo identificador está en la variable `$$`.
10. Si el identificador recibido no corresponde a ningún proceso vivo, el script termina con un mensaje que lo indique y con un código de salida distinto de cero. La comprobación se hace sobre la existencia del subdirectorio en `/proc`, antes de intentar leer nada.
11. Para el proceso consultado, el script informa el nombre, el estado, el identificador del padre y la cantidad de hilos. Los cuatro valores se leen del archivo `status`.
12. Cada línea de ese archivo trae la etiqueta, un separador y el valor:

```
$ grep Threads /proc/self/status
Threads: 1
```

El valor se extrae de la línea con un operador de patrones del shell, aplicado sobre la variable que la contiene. No se admite resolverlo invocando otro programa.

13. Ampliar el script para que, además del proceso consultado, recorra la cadena de padres con un ciclo hasta llegar al proceso 1, e imprima un renglón por cada proceso de la cadena.

14. El script termina con código de salida cero cuando el recorrido se completa.

Tercera parte: el mismo informe escrito con llamadas al sistema

La lectura con `read` no se hace por líneas. La llamada entrega bytes y devuelve cuántos entregó, sin atender a dónde terminan las líneas del archivo: puede devolver menos de lo pedido y continuar en la llamada siguiente donde quedó. Por esta razón el programa se organiza en dos etapas separadas.

En la primera etapa se lee el archivo completo a un arreglo de caracteres, con un ciclo que llama a `read` hasta que devuelva cero, acumulando lo de cada vuelta a continuación de lo que ya está en el arreglo. El valor de retorno indica cuántos bytes se agregaron.

La segunda etapa ocurre con el archivo ya cerrado. El arreglo se termina en el carácter nulo, con lo que su contenido pasa a ser una cadena, y localizar el valor de cada campo es entonces un problema de manejo de cadenas: la etiqueta aparece al comienzo de su línea, y el valor está entre el tabulador que la sigue y el fin de línea.

Los archivos de `/proc` se generan al leerlos, de modo que su tamaño no consta en ninguna parte: `stat` informa cero sobre ellos, y ese cero no es el tamaño del contenido sino la ausencia del dato. El arreglo se dimensiona entonces con holgura y el ciclo se detiene también cuando se llena. Si el contenido no cabe, el programa lo informa y termina con un código de error, en lugar de entregar un informe construido sobre una lectura incompleta.

15. Escribir un programa en C llamado `infoproc.c` que produzca la misma información del punto 11 sobre el proceso cuyo identificador reciba como argumento. Si no recibe argumento, informa sobre sí mismo.
16. **El archivo se abre con `open`, se lee con `read` y se cierra con `close`.** No se admite el uso de `fopen`, `fgets` ni ninguna otra función de la biblioteca de entrada y salida con formato. La restricción es sobre la lectura del archivo, no sobre el análisis del texto ya leído: las funciones de manejo de cadenas de `string.h` sí se pueden usar.
17. La salida se escribe con `printf` o con `write` sobre `STDOUT_FILENO`: las dos formas son admisibles. La restricción del punto anterior es sobre la lectura del archivo, no sobre la salida ni sobre el formateo de cadenas en memoria, de modo que `snprintf` también se puede usar.
18. Verificar el valor de retorno de cada llamada al sistema. Si una falla, informar la causa con `perror` y terminar con un código de error.
19. Informar también el identificador del propio programa y el de su proceso padre, obtenidos con `getpid` y `getppid`. Ejecutar el programa desde el shell y comprobar a qué proceso corresponde el padre.

20. El programa debe compilar sin advertencias:

```
$ gcc -Wall -o infoproc infoproc.c
```

Junto al enunciado se publica un Makefile que automatiza esa orden y agrega un objetivo clean que borra lo generado. Con él, construir y limpiar son:

```
$ make
$ make clean
```

El Makefile forma parte de lo que se entrega. Las órdenes de sus reglas van precedidas de un tabulador, que se pierde al copiar el texto desde este documento: se usa el archivo publicado.

Cuarta parte: la comparación

21. Ejecutar el script y el programa sobre el mismo identificador de proceso y comprobar que la información coincide.
22. Explicar en la bitácora de dónde sale la información en cada caso, y por qué dos programas tan distintos entregan el mismo resultado.
23. Ejecutar el script y el programa sobre el identificador de un proceso que esté cambiando de estado, como el navegador o un compilador en ejecución, y observar si el informe es idéntico. Explicar en la bitácora el resultado.
24. El shell también es un programa. Explicar en la bitácora qué hace el shell, en términos de servicios del sistema operativo, desde que se escribe una línea de comandos hasta que aparece de nuevo el símbolo de espera.

Quinta parte: una tarea de administración resuelta con un script

Se tiene una cantidad grande de videos en formato MKV que hay que subir a un servicio que prefiere MP4. El número de archivos hace impráctica la conversión manual.

Esta parte se resuelve desde la sesión del usuario creado en la preparación, y sus capturas de pantalla deben mostrar ese nombre en el símbolo del sistema.

25. Instalar la herramienta de conversión y comprobar que responde:

```
$ sudo apt install ffmpeg
$ ffmpeg -version
```

26. Generar los archivos de prueba con la misma herramienta, en lugar de descargar material. Cada archivo dura tres segundos y ocupa muy poco:

```
$ ffmpeg -f lavfi -i testsrc=duration=3:size=320x240 clase1.mkv
```

Generar al menos cuatro archivos, uno de ellos con varios puntos en el nombre, como `clase.2026.mkv`.

27. Escribir un script llamado `convertir.sh` que tome **todos** los archivos con extensión `.mkv` del directorio actual y genere para cada uno el archivo `.mp4` correspondiente, conservando el nombre. **El cambio de extensión se resuelve con un operador de patrones**, no invocando otro programa.
28. Hacer que el script no vuelva a convertir un archivo cuyo `.mp4` ya exista, e informe de que lo está saltando.
29. Aceptar como parámetro opcional el directorio donde buscar. Si no se recibe, usar el directorio actual. Si la ruta recibida no existe o no es un directorio, terminar con un mensaje claro y sin convertir nada.
30. Informar al final cuántos archivos se convirtieron y cuántos se saltaron.
31. Ejecutar el script dos veces seguidas sobre el mismo directorio. La segunda ejecución no debe convertir nada ni producir ningún error.
32. Registrar como evidencia tres capturas de pantalla, tomadas desde la sesión del usuario creado en la preparación y con el símbolo del sistema visible: la comprobación del punto 25, la primera ejecución del script y la segunda ejecución del punto 31. En una entrega en pareja, al menos una de las tres se toma desde la sesión de cada integrante.

Qué se entrega

- El script `infoproc.sh`, con permiso de ejecución y con la primera línea que declara el intérprete.
- El programa `infoproc.c`, compilando sin advertencias con `gcc -Wall`.
- El script `convertir.sh`, con las mismas condiciones del anterior.
- El `Makefile` publicado con el enunciado, que construye el programa.
- Las tres capturas de pantalla del punto 32, en las que se lee el nombre del usuario o de los usuarios creados.
- La bitácora de la práctica, con las anotaciones de los puntos 6, 8, 22, 23 y 24.

La entrega es individual o en pareja, y admite dos vías, que tienen el mismo valor. Un grupo no puede pasar de dos personas.

Por repositorio. Todo lo anterior registrado en el repositorio propio y subido al servidor remoto. En la actividad de Google Classroom se deja la dirección del repositorio. El trabajo que quede registrado únicamente en el equipo donde se hizo no está entregado.

Por archivo comprimido. Todo lo anterior dentro de un archivo .zip que se sube a la actividad de Google Classroom, armado como se indica en la sección siguiente.

Cómo se arma el archivo comprimido

El archivo se comprime *dentro de la máquina virtual*, desde la sesión del usuario creado en la preparación. Al copiar los archivos al sistema anfitrión y comprimirlos allí se pierde el permiso de ejecución de los scripts, de modo que el trabajo llega sin una de las condiciones que se piden.

- Reunir en un directorio, cuyo nombre sea el login institucional, los tres archivos de código, el Makefile, las tres capturas y la bitácora. En una entrega en pareja el directorio lleva los dos logins, separados por un guion bajo, y se sube una sola vez:

```
$ mkdir milogin
$ cp infoproc.sh infoproc.c convertir.sh Makefile milogin/
$ cp bitacora.txt captura1.png captura2.png captura3.png milogin/
```

- Comprimir el directorio completo. La herramienta se instala como la del punto 25:

```
$ sudo apt install zip
$ zip -r milogin.zip milogin
```

- Comprobar el contenido antes de subirlo. La lista debe mostrar los ocho archivos dentro del directorio, y no debe incluir los archivos de video generados en la quinta parte ni el ejecutable que produce el compilador:

```
$ unzip -l milogin.zip
```

Qué se valora

- Que los scripts y el programa comprueben antes de actuar: existencia del parámetro, existencia del proceso o de la ruta, y valor de retorno de cada llamada.
- Que la extracción del valor y el cambio de extensión usen operadores de patrones del shell, y no otro programa.
- Que el programa en C lea el archivo exclusivamente con llamadas al sistema.
- Que make construya el programa sin pasos manuales, y que make clean deje el directorio como estaba.
- Que el programa detecte el caso en que el contenido no quepa en el espacio reservado, y lo informe en lugar de entregar un informe incompleto con apariencia de correcto.

- Que la segunda ejecución del punto 31 sea silenciosa, y no un error controlado a la fuerza.
- Que las capturas correspondan a los usuarios creados en la preparación, con al menos una por cada integrante. Una captura tomada desde otra cuenta no acredita el trabajo de quien entrega.
- Las explicaciones de los puntos 22, 23 y 24: son las que distinguen haber entendido el mecanismo de haber obtenido una salida correcta.
- El uso de la documentación del sistema como fuente. Consultar man forma parte del trabajo.

Errores frecuentes

- Dimensionar el buffer de lectura con el tamaño que informa `stat` sobre el archivo. Los archivos de `/proc` informan tamaño cero porque su contenido se genera al leerlos.
- Suponer que `read` entrega el archivo completo en una sola llamada, y no usar la cantidad de bytes que devuelve.
- Tratar como cadena lo que entrega `read` sin terminarlo antes en el carácter nulo. La llamada entrega bytes y no agrega ningún terminador.
- Al escribir con `write`, pasar la cantidad de caracteres en lugar de la cantidad de bytes.
- Recorrer la cadena de padres sin condición de terminación. El proceso 1 es su propio límite: su campo `PPid` vale cero, y ese subdirectorío no existe.
- Rehacer el `Makefile` a mano copiando su texto de este documento. El tabulador que precede a cada orden se pierde al copiar, y `make` responde *missing separator*, que no nombra el espacio en blanco como causa.
- Suponer que el identificador de un proceso se mantiene entre ejecuciones.
- Confundir el proceso padre con la ventana de terminal desde la que se lanzó el programa.
- Suponer que el comodín `*` incluye los archivos ocultos. No lo hace, y no avisa.
- Construir el nombre de salida por concatenación en lugar de recortar la extensión. El resultado se evidencia con un archivo cuyo nombre contenga varios puntos, como `clase.2026.mkv`.
- Usar una variable sin comillas cuando el nombre del archivo puede tener espacios, o cuando su contenido puede venir vacío.

- Tomar las capturas antes de abrir la sesión del usuario creado. El símbolo del sistema muestra entonces otra cuenta y la evidencia no corresponde.
- Agregar el usuario al grupo sudo con una sesión suya ya abierta. La pertenencia a grupos se establece al iniciar la sesión, de modo que la sesión abierta antes del cambio sigue sin el permiso. Se resuelve cerrándola y volviéndola a abrir.
- Dar el trabajo por entregado después de registrar los cambios en el repositorio local, sin subirlos al servidor remoto.
- Armar el archivo comprimido en el sistema anfitrión, después de sacar los archivos de la máquina virtual. Los scripts llegan sin permiso de ejecución, y esa condición se pide en dos de los tres archivos de código.
- Comprimir los archivos sueltos en lugar del directorio que lleva el login. Al descomprimir, el contenido se mezcla con el del directorio de destino y deja de distinguirse a quién pertenece.
- Incluir en el archivo comprimido los videos generados en la quinta parte, o el ejecutable que produce el compilador. Ninguno de los dos forma parte de lo que se entrega.

